Building Durable Workflows in EDSL

Dependencies, human delivery, simulated respondents, derived values, repetition, and recovery

Expected Parrot

September 2026

Contents

Chapter 1

Purpose and scope

An EDSL workflow coordinates work performed by people, language-model agents, or a mixture of both. It decides who receives each task, when a task becomes available, which earlier answers it may observe, how results are combined, and what happens after failures or restarts.

The workflow layer is deliberately separate from delivery. Humanize can create private surveys, send invitations, and collect responses. A simulated inbox can deliver the same work to EDSL agents. The workflow coordinator remains the authority that decides what becomes ready next.

This manual describes the implementation in `edsl.workflows`. It covers:

- dependency graphs and work-item lifecycles;
- participant selection, fan-out, and fan-in;
- typed answer, output, and submission references;
- serializable conditions and deterministic derived values;
- bounded repeat blocks;
- visibility and anonymity boundaries;
- shared-state reads and writes;
- human delivery and LLM simulation;
- durable retries, leases, and process recovery;
- SQLite persistence and DAG visualization; and
- reusable design patterns and current limitations.

The workflow API is experimental. The serialized form and behavior described here are the current implementation, not a promise of permanent compatibility.

Chapter 2

The central idea

A workflow turns a declarative graph into durable, respondent-specific work:

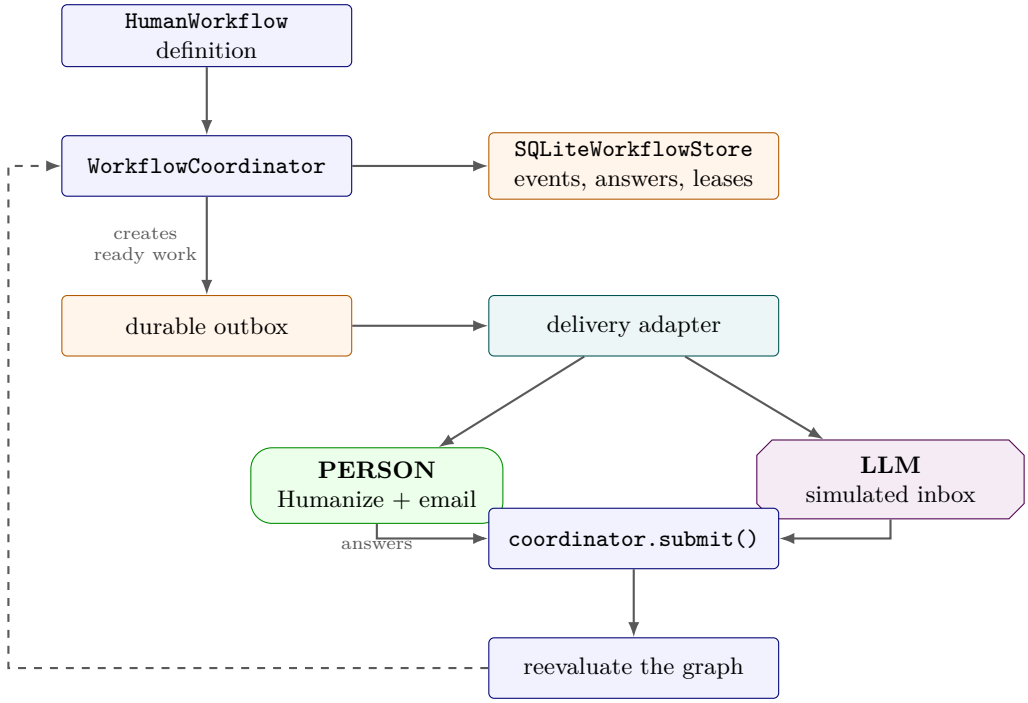

Figure 2.1: A workflow definition is evaluated by the coordinator, persisted in the store, and delivered through interchangeable human or LLM channels. Green rounded actors are people; violet clipped-corner actors are LLMs.

The key invariant is:

> Delivery systems do not decide workflow order. They deliver ready work and return answers. Only the coordinator evaluates dependencies and conditions.

This separation allows the same definition to run with real respondents during fieldwork and with LLM respondents during design, testing, and stress analysis.

2.1 Who performs the work?

The workflow definition says who is eligible for a step by selecting a participant role. It does **not** currently say whether that participant is a person or an LLM. That choice belongs to execution:

Execution path	Performer	Mechanism
Human fieldwork	**PERSON**	A Humanize delivery adapter emails a private survey and imports the response.
Simulation	**LLM**	`WorkflowSimulation` opens ready items and an `EDSLAgentAnswerer` runs their surveys with a model.
Deterministic test	**SCRIPT**	A scripted answerer supplies known responses without a person or model.

This neutrality is useful: one serialized protocol can be rehearsed with LLMs and later deployed unchanged to people. It also means that diagrams in this manual mark the **execution path**, not a property embedded in `HumanStep`. Green rounded actor nodes mean a person; violet clipped-corner actor nodes mean an LLM.

Performer routing is declared separately from the scientific workflow:

```
from edsl.workflows import ExecutionPlan, human, llm, role

plan = (
    ExecutionPlan()
    .bind(role("field-researcher"), human(channel="humanize-email"))
    .bind(role("coder"), llm(model_policy="research-coder"))
)
```

The plan round-trips through `to_dict()` and `from_dict()`, requires exactly one matching binding per participant, and lets `WorkflowSimulation` select distinct answerers by executor kind. Production still needs an adapter registry that dispatches the human bindings to Humanize and persists the resolved executor on each attempt. Step metadata may describe the intended performer for diagrams, but it is not the routing authority.

2.2 Waiting for optional work

Normal `after=` dependencies propagate a predecessor's skip. Use `after_settled=` when a downstream step must wait for optional work but should still run if that work is skipped:

```
builder.step(
    "draft-report",
    report_survey,
    assigned_to=role("report-writer"),
    after_settled=human_adjudication,
)
```

Both dependency modes wait until every predecessor item is completed or skipped. Only `after=` treats a skipped predecessor as a reason to skip the consumer.

Chapter 3

A first workflow

Suppose one person proposes a weekend activity and a second person approves it.

```
from edsl import Agent, QuestionFreeText, QuestionYesNo, Survey
from edsl.workflows import Workflow, role

builder = Workflow("Weekend activity approval")

activity = QuestionFreeText(
    question_name="activity",
    question_text="Suggest a weekend activity.",
)
proposal = builder.step(
    "propose",
    Survey([activity]),
    assigned_to=role("proposer"),
)

approved = QuestionYesNo(
    question_name="approved",
    question_text=(
        "The proposed activity is "
        f"{proposal.answer(activity).template}. Do you approve?"
    ),
)
builder.step(
    "approve",
    Survey([approved]),
    assigned_to=role("approver"),
    after=proposal,
)

workflow = builder.compile()
```

The second task is not merely listed after the first in Python. Its serialized definition contains an explicit dependency on `propose`. The question also contains a typed reference to the proposal answer.

Participants are ordinary EDSL agents whose traits determine their roles:

```
participants = [
    Agent(
        name="proposer@example.com",
```

```
        traits={"role": "proposer", "email": "proposer@example.com"},
    ),
    Agent(
        name="approver@example.com",
        traits={"role": "approver", "email": "approver@example.com"},
    ),
]
```

Chapter 4

Definitions, instances, steps, and work items

Four terms should remain distinct.

4.1 Workflow definition

A `HumanWorkflow` is immutable, serializable data. It contains steps, metadata, derived-value declarations, and repeat-block declarations. It does not contain a database connection, model client, active respondent, or Python callback.

4.2 Workflow instance

Launching a definition creates one instance with a unique identifier, a saved copy of the definition, a participant roster, and an execution status.

```
store = SQLiteWorkflowStore("workflow.sqlite")
coordinator = WorkflowCoordinator(workflow, store)
instance_id = coordinator.launch(participants)
```

4.3 Step

A `HumanStep` describes one kind of task: its survey, assignee selector, dependencies, enable condition, completion policy, visibility, and optional shared-state operations.

4.4 Work item

A work item is the assignment of one step to one participant in one instance. If a step selects six experts, launch creates six work items. The step is the graph node in the definition; the work items are its respondent-specific realizations.

Chapter 5

Work-item lifecycle

The normal lifecycle is:

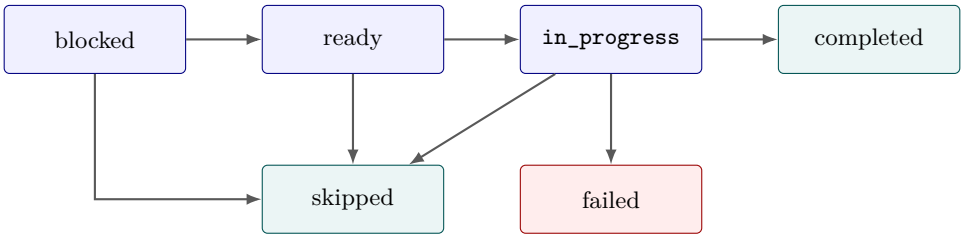

Figure 5.1: Work-item states are identical for human- and LLM-performed work. Completed, skipped, and failed are terminal.

`blocked` The dependencies have not reached terminal states.
`ready` Dependencies are terminal and the enable condition is true. A durable outbox record exists for delivery.
`in_progress` The survey was opened or an execution attempt acquired a lease.
`completed` A valid answer was submitted exactly once.
`skipped` A dependency was skipped, a condition evaluated false, quorum superseded the assignment, or a repeat terminated before this iteration.
`failed` The configured retry budget was exhausted or an error was classified as non-retryable.

A dependency is settled when all of its work items are **`completed`** or **`skipped`**. A failed item instead fails its workflow instance.

Chapter 6

Participant selection and fan-out

`role(name)` constructs an exact trait selector:

```
review = builder.step(
    "review",
    review_survey,
    assigned_to=role("reviewer"),
)
```

Every participant with `traits["role"] == "reviewer"` receives an independent work item. This is fan-out.

The underlying selector can match several traits:

```
from edsl.workflows import ParticipantSelector

senior_economists = ParticipantSelector(
    {"role": "reviewer", "discipline": "economics", "seniority": "senior"}
)
```

Matching is exact and conjunctive. There is not yet an expression language for inequalities, membership, or arbitrary roster queries.

Every step must match at least one participant at launch. Duplicate participant names are rejected because the name is the durable participant identifier.

Chapter 7

Dependencies and fan-in

Pass a step handle through `after`:

```
draft = builder.step("draft", draft_survey, assigned_to=role("author"))
review = builder.step(
    "review",
    review_survey,
    assigned_to=role("reviewer"),
    after=draft,
)
```

Multiple prerequisites form an AND dependency:

```
builder.step(
    "release",
    release_survey,
    assigned_to=role("publisher"),
    after=(legal_review, technical_review),
)
```

When a dependency has many assigned participants, the dependent step waits for the dependency's completion policy. The default is all assigned participants. This produces fan-in without a separate join node.

Conditions automatically add the steps they reference to `after`. Explicit `after` declarations are still recommended when they explain the graph.

Chapter 8

Typed references

Typed references reduce fragile, handwritten Jinja paths.

8.1 One answer

Use `answer()` when a source step has one relevant submission:

```
draft.answer("copy").template
```

This renders a path under `workflow.answers`.

8.2 All answers from a fan-out step

Use `outputs()` when every submission matters:

```
reviews.outputs("decision").template
```

This renders a list of values under `workflow.outputs`.

8.3 Identity-preserving submissions

Some mechanisms need both identity and answers:

```
reports.submissions.template
```

The rendered structure is conceptually:

```
[
    {
        "participant_id": "respondent-1",
        "answers": {"signal": "Red", "forecast": 60},
    },
    # ...
]
```

Use this only when identity is genuinely required. Anonymous synthesis should usually consume `outputs()` instead.

8.4 Fallback references

A branch may produce either an original or revised artifact:

```
latest = revision.answer("copy").template_or(draft.answer("copy"))
```

The fallback is evaluated during survey rendering. It does not execute Python at workflow runtime.

Chapter 9

Conditions and branching

Conditions are serializable predicates. A step runs only when its `when` condition evaluates true after all condition dependencies settle.

9.1 Answer equality

```
approved = review.answer("approved").equals("Yes")

builder.step("publish", ..., when=approved)
builder.step("revise", ..., when=not_(approved))
```

`if_(condition)` returns complementary `then` and `otherwise` conditions:

```
branch = if_(approved)
builder.step("publish", ..., when=branch.then)
builder.step("revise", ..., when=branch.otherwise)
```

9.2 Boolean composition

```
all_of(legal_ok, technical_ok)
any_of(editorial_ok, emergency_override)
not_(rejected)
join_all(a, b)
join_any(a, b)
```

`all_of` and `any_of` require at least one argument.

9.3 Aggregate predicates

```
votes.outputs("choice").count("Approve").at_least(2)
votes.outputs("choice").has_disagreement
votes.outputs("choice").majority_is("Approve")
estimates.outputs("value").range_at_most(10)
```

9.4 Stable chance

```
chance(0.70, key="continue-after-round-2")
```

Chance is deterministic for one `(instance_id, key)` pair. Reevaluation and process restart therefore cannot change the draw. Always choose a stable, meaningful key.

Chapter 10

Completion policies and quorum

The default completion policy is `all_assigned()`.

For early fan-in, use quorum:

```
votes = builder.step(
    "moderator-vote",
    vote_survey,
    assigned_to=role("moderator"),
    completion=quorum(2),
)
```

Once two submissions complete, remaining work items for that step are skipped as superseded. Downstream work may then proceed. A quorum larger than the number of matching participants is rejected during launch.

Quorum does not presently express weighted votes, role-specific quotas, or deadline-based partial completion.

Chapter 11

Deterministic derived values

Language models should interpret evidence, not perform authoritative arithmetic. Derived values provide a serializable calculation layer modeled on the shared-state expression DSL.

```
stats = builder.derive(
    "round-2-statistics",
    mean=round_2.outputs("estimate").mean(),
    median=round_2.outputs("estimate").median(),
    minimum=round_2.outputs("estimate").minimum(),
    maximum=round_2.outputs("estimate").maximum(),
    spread=round_2.outputs("estimate").range(),
)
```

Reference a field in a prompt:

```
QuestionFreeText(
    question_name="summary",
    question_text=(
        f"The authoritative mean is {stats.field('mean').template}; "
        f"the range is {stats.field('spread').template}. Interpret them."
    ),
)
```

Or use a derived field in a condition:

```
converged = stats.field("spread").at_most(10)
```

Derived fields may reference earlier derived fields:

```
outcome = builder.derive(
    "round-2-outcome",
    converged=stats.field("spread").expression.compare_at_most(10),
)
```

The resulting expression tree contains named, allowlisted operators. It has no callable, module path, source code, `eval`, or lambda. This is invalid:

```
builder.derive("unsafe", mean=lambda values: sum(values) / len(values))
```

The builder rejects it because derived fields must be `WorkflowExpression` objects.

11.1 Expression serialization

A mean expression resembles:

```
{
  "type": "workflow_expression",
  "op": "mean",
  "args": [
    {
      "type": "workflow_expression",
      "op": "step_outputs",
      "options": {
        "step_name": "round-2",
        "question_name": "estimate"
      }
    }
  ],
  "options": {}
}
```

Deserialization rejects unknown operators. Dependencies are extracted from the tree and checked against the workflow. Derived references must name an earlier definition and an existing field.

11.2 Supported expression operations

The current evaluator supports:

```
Sources:      step_outputs, derived_ref
Aggregates:   mean, median, minimum, maximum, range
Arithmetic:   add, subtract, multiply, divide
Comparisons:  at_most, at_least, equals
```

Missing source outputs delay availability. Non-numeric values cause numeric aggregates to fail rather than silently inventing coercions beyond `float()`.

Chapter 12

Economic settlement primitives

Workflow expressions can consume a single answer or identity-preserving submissions. This supports authoritative arithmetic without asking a model to calculate payoffs:

```
accepted = response.answer("accept").value.compare_equals("Yes")
payoffs = builder.derive(
    "payoffs",
    proposer=choose(accepted, 10 - offer.answer("offer").value, 0),
    responder=choose(accepted, offer.answer("offer").value, 0),
)
```

Two-player normal-form games use a serializable payoff matrix. Explicit action codes avoid ambiguity when labels share an initial letter; matrix keys concatenate the codes in stable participant-ID order:

```
payoffs = choices.submissions.payoff_matrix(
    "action",
    {"WW": (2, 2), "WT": (1, 3), "TW": (3, 1), "TT": (0, 0)},
    action_codes={"Swerve": "W", "Straight": "T"},
)
```

Codes must be unique one-character strings. Omitting `action_codes` retains the legacy first-character convention for serialized workflows created before this option was introduced.

`closest_to(question, target, ties="all")` performs identity-aware ranking. `DerivedFieldRef.for_participant()` projects an identity-keyed mapping into a private recipient prompt. `match(role("player"), size=2)` deterministically partitions a roster and round-trips as data.

Dynamic answer bounds are enforced when an answer is submitted:

```
builder.step(
    "return",
    return_survey,
    after=sent,
    answer_bounds={returned_question: (0, sent.answer("sent").value * 3)},
)
```

Static survey bounds remain useful for presentation; dynamic bounds are the coordinator's authoritative validation rule.

Chapter 13

Bounded repetition

Many workflows are conceptually loops: Delphi rounds, revision cycles, negotiations, repeated games, and retry-until-accepted tasks.

Use a repeat block to avoid manually defining every round:

```
def build_round(iteration):
    estimate = QuestionNumerical(
        question_name="estimate",
        question_text=f"Round {iteration.number}: give your estimate.",
        min_value=0,
        max_value=100,
    )
    forecast = iteration.step(
        "round",
        Survey([estimate]),
        assigned_to=role("expert"),
    )
    iteration.stop_when(
        forecast.outputs("estimate").range().at_most(10)
    )

rounds = builder.repeat(
    "delphi-rounds",
    min_iterations=2,
    max_iterations=3,
    build=build_round,
)
```

The `build` callable is authoring sugar. It runs while constructing the definition and is not serialized. The compiled `HumanWorkflow` contains a `RepeatBlock` with bounds, iteration numbers, materialized step names, typed stop conditions, and repeat metadata on each step.

This distinction is important:

- bounded iterations are materialized during authoring;
- the persisted definition contains only data;
- execution after deserialization does not need `build_round`; and
- future iterations are skipped when the prior stop condition succeeds.

`min_iterations` forces early rounds to run even if their stop conditions are already true. `max_iterations` is a hard safety bound.

The current implementation is not an unbounded runtime loop. Large or data-dependent iteration counts will eventually require runtime materialization of a serializable subworkflow template.

Chapter 14

Output visibility and confidentiality

By default, a step's outputs are available to downstream assignees. Restrict them with `visible_to`:

```
sealed = builder.step(
    "sealed-bid",
    bid_survey,
    assigned_to=role("vendor"),
    visible_to=role("buyer"),
)
```

Several roles may be authorized:

```
visible_to=(role("expert"), role("facilitator"))
```

During compilation, EDSL rejects a typed reference when the consumer's selector can never satisfy the source visibility policy. Visibility also propagates through derived values: computing a mean from sealed bids does not make that mean public automatically.

This is a definition-level information-flow check, not a complete security system. Operational deployments must also secure SQLite files, application logs, email content, Humanize access, credentials, and administrator tooling.

An explicit anonymity policy with pseudonyms and identity audit rules remains a future extension. Today, anonymous synthesis is expressed by sending answer values without `participant_id` and limiting source visibility.

Chapter 15

Shared state inside workflows

Dependencies move information between steps. Shared state provides durable, typed resources that several steps may read or update.

Two convenience resources are available:

```
from edsl.workflows import Artifact, Collection

submission = Artifact("paper-submission")
reviews = Collection("paper-reviews")
```

An artifact is a write-once text value. A collection is an append-only sequence of actor/value records. Both are ordinary `SharedStateMap` definitions.

A step declares its accesses:

```
paper = submission.by("paper-1").artifact
review_log = reviews.by("paper-1").collection

builder.step(
    "submit",
    submission_survey,
    assigned_to=role("author"),
    writes=(paper.submit(value=submission_question.answer),),
)

builder.step(
    "review",
    review_survey,
    assigned_to=role("reviewer"),
    after=submit,
    reads=(paper.read(),),
    writes=(
        review_log.add(
            actor=current.agent.name,
            value=review_question.answer,
        ),
    ),
)
```

The coordinator renders reads into `shared_state`, records the observed state versions, applies writes after answer submission, and rejects a submission if a shared-state command rejects its transition.

Shared-state semantics are documented in `docs/shared_state_dsl_manual.md` and the normative material under `docs/shared_state_semantics/`.

Chapter 16

Persistence with SQLite

Create a local durable store:

```
store = SQLiteWorkflowStore("runs/my-workflow/workflow.sqlite")
```

The store maintains these logical records:

Record	Purpose
`workflow_instances`	Definition snapshot and overall status
`workflow_participants`	Serialized EDSL agents
`workflow_items`	Per-participant task lifecycle
`workflow_submissions`	Idempotent answers
`workflow_events`	Ordered audit history
`workflow_outbox`	Durable delivery requests
`workflow_external_tasks`	Humanize/provider identifiers
`workflow_item_renders`	Exact rendered survey and state snapshot
`workflow_attempts`	Attempt number, lease, outcome, and error class

The database is initialized additively. Opening an older workflow database creates newly introduced tables without deleting existing execution data.

Chapter 17

Opening and submitting work

`coordinator.open(item_id)`:

1. verifies that the item is ready or in progress;
2. loads participant traits;
3. reads authorized shared state;
4. assembles prior visible answers and submissions;
5. evaluates available derived values;
6. renders every survey question;
7. saves the exact render and state versions; and
8. marks the item in progress.

The returned `OpenedWorkItem` contains the rendered survey and execution context.

Submit an answer with a stable idempotency key:

```
coordinator.submit(
    item_id,
    {"approved": "Yes"},
    idempotency_key="humanize:survey-response-uuid",
)
```

Submitting the same key for the same item twice produces one durable submission and one completion event. Reusing it for a different item is not accepted as a valid completion.

Chapter 18

The durable outbox

When an item becomes ready, the coordinator creates an outbox record in the same workflow store. Delivery reads only pending records:

```
dispatcher = OutboxDispatcher(store, adapter)
receipts = dispatcher.dispatch()
```

Each `DeliveryRequest` contains:

```
idempotency_key
instance_id
work_item_id
step_name
participant_id
```

An adapter must use the idempotency key when its external system supports one. The outbox protects the ready decision from a process failure between graph evaluation and delivery.

The current dispatcher marks delivery after the adapter returns. Production adapters must themselves be idempotent because a crash after external delivery but before the local mark can cause redelivery.

Chapter 19

Humanize and real respondents

`HumanizeDeliveryAdapter` is intentionally thin. For each ready item it:

1. opens and renders the participant-specific survey;
2. creates a private Humanize survey;
3. creates a delivery;
4. records the external survey and delivery identifiers; and
5. returns a delivery receipt.

```
adapter = HumanizeDeliveryAdapter(
    coordinator,
    subject_prefix="Weekend activity",
)
OutboxDispatcher(store, adapter).dispatch()
```

Later, polling imports completed responses:

```
completed_count = adapter.poll_completed()
```

For each response, the adapter calls `coordinator.submit()`. That submission may make new items ready, after which the outbox dispatcher delivers the next wave.

Humanize owns survey presentation, invitations, and response collection. It does not need to understand the dependency graph.

Chapter 20

Simulated respondents

The simulation uses the same coordinator and store with an in-memory inbox:

```
from edsl import Model
from edsl.workflows import EDSLAgentAnswerer, WorkflowSimulation

answerer = EDSLAgentAnswerer(
    Model("gpt-4o-mini", service_name="openai"),
    run_options={"disable_remote_inference": False},
)

simulation = WorkflowSimulation(
    coordinator,
    {agent.name: agent for agent in participants},
    answerer,
)
simulation.run(instance_id)
```

`EDSLAgentAnswerer` runs each rendered survey through the ordinary EDSL agent/model pipeline. A scripted object implementing `answer(agent, opened)` is often better for deterministic tests.

`response_delay` and the virtual clock can model elapsed time without sleeping:

```
simulation = WorkflowSimulation(
    coordinator,
    agents,
    answerer,
    response_delay=timedelta(hours=2),
)
```

Chapter 21

Durable attempts and recovery

Remote inference and external delivery fail in several distinguishable places: submission, model execution, result retrieval, process shutdown, and network timeout. A durable workflow must resume rather than restart completed work.

Configure simulation recovery with `RetryPolicy`:

```
from edsl.workflows import RetryPolicy

simulation.run(
    instance_id,
    resume=True,
    retry_policy=RetryPolicy(
        max_attempts=3,
        lease_seconds=300,
        retryable=("remote_error", "timeout", "exception"),
    ),
)
```

21.1 Attempt lifecycle

Before an answerer runs, the store creates a numbered attempt with a lease:

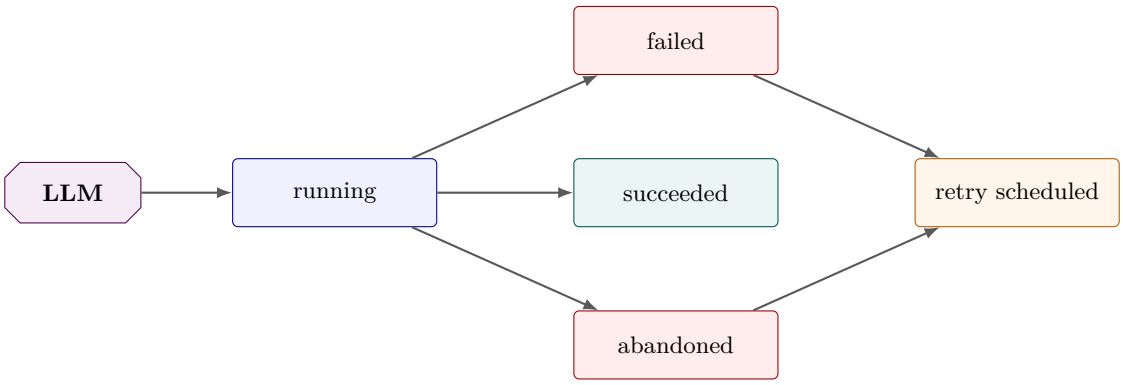

Figure 21.1: During LLM simulation, a durable answerer attempt either succeeds or records why another attempt may be scheduled. Human delivery has its own durable outbox and external-delivery records.

Every attempt records its start, lease expiration, finish, status, error kind, and a bounded error message.

21.2 Leases

A lease prevents two healthy workers from claiming the same in-progress item. If the process disappears, another process can reclaim the item after the lease expires. Recovery marks the old attempt `abandoned` with `error_kind="lease_expired"` before scheduling the next attempt.

Choose a lease comfortably longer than ordinary answer latency. Automatic lease heartbeats are not yet implemented.

21.3 Error classification

The simulator currently classifies:

`timeout` Python `TimeoutError`.
`remote_error` Exception type identities containing `remote` or `coop`.
`exception` Other exceptions.

Only listed classes are retried. Classification is intentionally conservative and should eventually become a typed exception protocol rather than a naming heuristic.

21.4 Retry exhaustion

After `max_attempts`, the item becomes `failed` and the workflow instance becomes `failed`. The simulator raises a `RuntimeError` naming the failed steps. Failed instances do not silently masquerade as completed runs.

21.5 Restarting in a fresh process

Reconstruct the definition and coordinator, then resume against the same file:

```
restored = HumanWorkflow.from_dict(saved_definition)
store = SQLiteWorkflowStore("workflow.sqlite")
coordinator = WorkflowCoordinator(restored, store)
simulation = WorkflowSimulation(coordinator, agents, answerer)
simulation.run(instance_id, resume=True, retry_policy=policy)
```

Recovery requeues:

- ready items whose earlier delivery record is no longer pending;
- in-progress items with no active attempt; and
- items whose running attempt lease expired.

It does not reclaim an item with an unexpired lease.

Chapter 22

Serialization and portability

Round-trip a workflow with:

```
payload = workflow.to_dict()
restored = HumanWorkflow.from_dict(payload)
assert restored.to_dict() == payload
```

The serialized object includes:

```
{
  "type": "human_workflow",
  "version": 1,
  "name": "Example",
  "steps": [],
  "metadata": {},
  "derived_values": [],
  "repeat_blocks": []
}
```

Surveys, selectors, conditions, completion policies, visibility selectors, state reads and writes, derived expressions, and repeat declarations all round-trip as data.

Execution policy such as a `RetryPolicy` is separately serializable but is not currently embedded in `HumanWorkflow`. This lets one deployment use a different operational retry budget without changing the research design.

Chapter 23

Visualization and audit evidence

Render an instance as a standalone vertical DAG:

```
from edsl.workflows import WorkflowDAGVisualization

path = WorkflowDAGVisualization(coordinator, instance_id).save("dag.html")
```

The visualization shows:

- wall-clock ordering on the vertical axis;
- dependency edges;
- participant-specific colors;
- question-type glyphs;
- work-item status;
- rendered question text and options;
- submitted responses;
- shared-state snapshots and transitions;
- lifecycle timestamps;
- execution attempts and error classifications; and
- the ordered workflow event timeline.

Failed items are visually distinct. Repeat conditions and derived expression conditions are rendered as readable labels.

The visualization reads persisted evidence. It does not rerun conditions or call a model merely to construct the page.

Chapter 24

Pattern: parallel brainstorm and selection

```
ideas = builder.step(
    "idea",
    idea_survey,
    assigned_to=role("ideator"),
)

builder.step(
    "select",
    Survey([
        QuestionFreeText(
            question_name="choice",
            question_text=f"Choose from {ideas.outputs('idea').template}.",
        )
    ]),
    assigned_to=role("chair"),
    after=ideas,
)
```

This is the simplest fan-out/fan-in pattern.

Chapter 25

Pattern: blind review

An author writes an artifact. Several reviewers can read it but cannot read one another's reviews. An editor receives the collected reviews.

Use output visibility for sealed responses and shared-state capabilities for durable artifacts. Be explicit about which layer protects which information.

Chapter 26

Pattern: approval with revision

```
branch = if_(review.answer("approved").equals("Yes"))

revision = builder.step("revise", ..., when=branch.otherwise)
builder.step(
    "publish",
    Survey([
        QuestionFreeText(
            question_name="publication",
            question_text=(
                "Publish "
                f"{revision.answer('copy').template_or(draft.answer('copy'))}"
            ),
        )
    ]),
    after=(review, revision),
    when=join_any(branch.then, revision.completed),
)
```

This pattern demonstrates complementary branches and typed fallback selection.

Chapter 27

Pattern: Delphi forecasting

A Delphi workflow combines several features:

1. experts answer independently;
2. only the facilitator sees identity-bearing source work;
3. the facilitator returns an anonymous synthesis;
4. derived expressions calculate authoritative statistics;
5. a bounded repeat continues for at least two rounds; and
6. a typed range condition stops later rounds after convergence.

The simulation that motivated derived values revealed an important principle: the facilitator correctly discussed disagreements but misstated the arithmetic and termination result. The engine should compute facts; the model should explain them.

Chapter 28

Pattern: probabilistically ending public-goods game

```
def build_round(iteration):
    previous = rounds_by_number.get(iteration.number - 1)
    # Construct a survey referencing the previous contribution vector.
    current = iteration.step(
        "round",
        survey,
        assigned_to=role("player"),
    )
    iteration.stop_when(
        not_(chance(0.70, key=f"continue-after-round-{iteration.number}"))
    )
    rounds_by_number[iteration.number] = current

builder.repeat("public-goods-rounds", max_iterations=5, build=build_round)
```

Stable chance ensures a restart cannot redraw whether the game continues.

Chapter 29

Pattern: peer prediction

Peer prediction requires identity-preserving sealed reports, deterministic matching, and deterministic scoring. The current workflow can deliver sealed submissions to a scorer, but scoring arithmetic should be a derived expression or a future allowlisted algorithm—not prose delegated to an LLM.

Per-recipient output projection is also important: respondents should receive their own score rather than the full scoring table.

Chapter 30

Testing workflows

Use scripted answerers for semantic tests:

```
class Answers:
    def answer(self, agent, opened):
        if opened.step_name == "propose":
            return {"activity": "Sailing"}
        return {"approved": "Yes"}
```

A strong workflow test should verify:

- definition serialization and reconstruction;
- initial ready fan-out;
- delivery order;
- exact rendered downstream context;
- completed and skipped work-item statuses;
- condition decisions;
- shared-state results and versions;
- submission idempotency;
- restart behavior using a new store and coordinator object;
- retry exhaustion and workflow failure; and
- visualization generation.

For derived expressions, also test rejection of raw Python and unknown operators. For visibility, test that an unauthorized derived reference fails compilation.

Chapter 31

Operational checklist

Before a real deployment:

1. Serialize and reconstruct the workflow in a fresh process.
2. Verify every role matches the intended participant count.
3. Audit every `visible_to` declaration and identity-bearing submission.
4. Use stable idempotency keys in delivery and submission adapters.
5. Put the SQLite database on durable, access-controlled storage.
6. Choose lease durations longer than expected provider latency.
7. Restrict retryable errors and set a finite attempt budget.
8. Test a crash after delivery and a crash during answer retrieval.
9. Use deterministic expressions for arithmetic and authoritative decisions.
10. Render and inspect the DAG and event history.
11. Verify Humanize surveys are private where required.
12. Decide how operators are alerted when an instance fails.

Chapter 32

Current limitations

The following are important but not yet first-class:

- runtime or unbounded repeat materialization;
- lease heartbeat and active renewal;
- a typed provider-error protocol;
- automatic cancellation of external work after quorum or branch supersession;
- deadlines, reminders, priorities, and escalation schedules;
- explicit anonymous-panel and pseudonym policies;
- per-recipient projection of a batch result;
- persisted snapshots of every derived value and condition evaluation;
- weighted quorum and richer participant selectors;
- deterministic quantiles, variance, standard deviation, and frequency tables;
- one `on_termination` step for every possible repeat exit; and
- transactional atomicity spanning workflow and separate shared-state databases.

These limitations are design boundaries, not invitations to hide behavior in Python callbacks or LLM prompts. New capabilities should extend the serializable DSL and coordinator evaluator.

Chapter 33

Compact API reference

33.1 Authoring

```
Workflow(name, metadata=None)
builder.step(name, survey, assigned_to=..., after=..., when=...,
             completion=..., visible_to=..., reads=..., writes=..., metadata=...)
builder.derive(name, **workflow_expressions)
builder.repeat(name, min_iterations=1, max_iterations=N,
               build=authoring_callback, after=None)
builder.compile() -> HumanWorkflow
```

33.2 Selectors and policies

```
role(name)
ParticipantSelector(mapping)
all_assigned()
quorum(count)
```

33.3 Conditions

```
answer.equals(value)
step.completed
all_of(*conditions)
any_of(*conditions)
not_(condition)
if_(condition)
chance(probability, key=stable_key)
outputs.count(value).at_least(count)
outputs.has_disagreement
outputs.majority_is(value)
outputs.range_at_most(maximum)
derived_field.at_most(value)
derived_field.at_least(value)
derived_field.equals(value)
```

33.4 Execution

```
SQLiteWorkflowStore(path)
WorkflowCoordinator(workflow, store, state_backends=None)
```

```
coordinator.launch(participants, instance_id=None)
coordinator.open(work_item_id)
coordinator.submit(work_item_id, answers, idempotency_key=...)
OutboxDispatcher(store, adapter).dispatch()
WorkflowSimulation(coordinator, agents, answerer).run(
    instance_id,
    resume=False,
    retry_policy=RetryPolicy(),
)
```

33.5 Inspection

```
store.items(instance_id)
store.item_answers(work_item_id)
store.step_answers(instance_id, step_name)
store.events(instance_id)
store.attempts(work_item_id)
WorkflowDAGVisualization(coordinator, instance_id).save(path)
```

Chapter 34

Closing principle

A trustworthy workflow keeps four responsibilities separate:

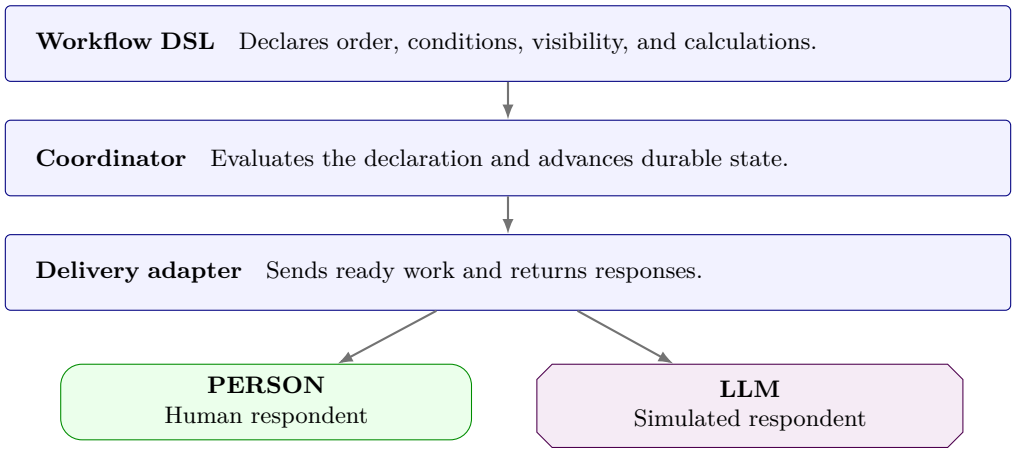

Figure 34.1: The responsibility boundaries in a workflow deployment. The delivery path terminates in either a person or an LLM; the workflow definition itself is the same.

When arithmetic, routing, retries, or confidentiality matter, they belong in the first two layers. Human and LLM respondents should receive well-defined work, not responsibility for enforcing the protocol that assigned it.

Chapter 35

Rebuilding this manual

The Markdown source is converted to standalone LaTeX and then compiled with XeLaTeX:

```
pandoc docs/workflow_manual.md \
  --standalone --to=latex \
  --output=docs/workflow_manual.tex

xelatex -interaction=nonstopmode -halt-on-error \
  -output-directory=docs docs/workflow_manual.tex
xelatex -interaction=nonstopmode -halt-on-error \
  -output-directory=docs docs/workflow_manual.tex
```

The second XeLaTeX pass resolves the table of contents and page references.